

A formal description of the type system

notch1p

August 21, 2026

1 Introduction

The purpose of this document is to provide readers with a up-to-date formal description of Tigris' type system, with a heavy emphasis on its implementation – judgements are all algorithmic, specifically. It is advised that this be read alongside all the notes¹ in **typing/** to form a more complete understanding of “how we did it”, “why it is done this way”. Meanwhile, System F elaboration is trivial modulo instance resolution and dictionary synthesis – since the description is not about the latter, we omit the former here completely.

For an *much outdated* review and formal description of the language, I suggest reading `doc/midterm` (in English) for a work-in-progress yet self-contained report, or my B.Eng. thesis² (in Chinese) located in the repo (and the submodule in `doc/`) **beng-thesis**. Directories **examples/** and **tests/cases/** showcase some of the (mostly) up-to-date features and capability of the language.

Layout. We don't follow the traditional formal grammar - judgements order but instead gives every important datatype their own sections.

Below we present the core calculus at both term and type level. Not everything is presented here, for brevity. Refer to **syntax.ott** for the complete syntaxes.

2 Terms

<i>expr, e</i>	<i>term</i>
$::=$	
<i>tname</i>	variable
$\lambda x. e$	lambda abstraction
let $x = e$ in e'	letexp
$e_1 e_2$	function application
(e_1, e_2)	product
if e_1 then e_2 else e_3	conditional
fix e	fixpoint
match e with $Pmatrix$	pattern matching
$e : sch$	type ascription

let-expressions and toplevel bindings have been simplified to take single binding. Same applies to case analysis. A recursive binding is no different from a normal one except its **fun**-shaped RHS is wrapped in a **fix** node with its name as the head argument; Others are processed as-is. Literals are omitted.

¹Readers are also assumed familiarity with the various works this document quotes, for being the sturdy foundation that Tigris relies on.

²TinyTheorem: A dependently-typed programming language with light theorem proving (I really should name it *Tigris* in the title.)

The Let(rec)-group in the surface language is inferred in SCC order, thus a nonrecursive binding may still be mutually recursive iff each one of them does not mention itself. Note that this is different from OCaml’s Let-group semantics.

3 Types

$tyvar, tv$	$::=$	type variables
	α	unification variable
	a	skolem (rigid)
$typeexpr, ty, \tau$	$::=$	type expression
	$tyctor$	nullary type constructor
	tv	type variable
	$\tau_1 \rightarrow \tau_2$	arrow type
	$\tau_1 \times \tau_2$	product type
	$\tau \overline{\tau_i}^i$	spinal, normal type application
	$\Lambda tv. ty$	type lambda
	sch	embedded scheme i.e. polytypes
$scheme, sch, \sigma$	$::=$	scheme
	$\forall \overline{tv_i}^i. \bar{\pi} \Rightarrow ty$	
$predicate, pred, \pi$	$::=$	predicate
	$tyctor\ ty_1 .. ty_i$	

Tigris is in System F. The type system supports arbitrary ranked types, allows polytypes to nest everywhere in the types³. Later we’ll show how *partial* impredicativity is obtained “for free” without Quick Look (Serrano et al. ICFP’20), unlike GHC – and why much of what we once boasted was unsound.

The usually-absent type lambda is part of the syntax directly to encode both higher-kinded types and type synonyms (abbreviations). It is handled with care to ensure no beta-redexes survive construction whereas partial applied type constructors get eta-expanded. Unlike GHC, we permit partially applied type synonyms.

Typeclasses in the surface language is syntactic sugar (with additional information recorded) for structures, which, in turn, is syntactic sugar for single-constructor inductive types whose type constructor’s *name* is the same as the value constructor. This invariant is enforced by the parser. A predicate is a typeclass constraint. Unlike GHC, typeclass constructor (often higher-rank) can be called directly and the methods can be explicitly pattern matched out of a instance with generalization.

We now describe expression typing. Note that toplevel binding is omitted since in practice we simply wrap them in a Letexp whose body is Unit.

3.1 Constraint Generation.

Tigris implements a constraint solver with wanted pools similar to that of OutsideIn(X) based on the understanding that type inference is a process of generating and solving constraints (Pottier, Rémy).

Remark. We deem a judgement *most* “algorithmic” if it generates constraints. Those judgements uses $\models$ whereas others use $\vdash$ unless told otherwise. Readers should not assume judgements are “pure” since most of them carry a state for e.g. counters used for fresh type variables.

³Though a higher-rank scheme cannot carry predicates, for now.

<i>constraint, cst</i>	::=	constraint
		$ty_1 \sim ty_2$ equality constraint
		$pred^{evidx}$ predicate constraint with evidence

Term Inference. $R; \Gamma \models e \Rightarrow ty \mid Cs$ reads, under typing environment Γ and rigid set R , e is inferred to be ty . Below we give the trivial ones directly, then elaborate on more interesting cases.

$R; \Gamma \models e \Rightarrow ty \mid Cs$		<i>(term inference)</i>
I-VAR $\frac{x : sch \in \Gamma \quad \models^i sch \Rightarrow ty_1 \mid Cs_1}{R; \Gamma \models x \Rightarrow ty_1 \mid Cs_1}$	I-LAM $\frac{\alpha \text{ fresh} \quad (R \cup \{\alpha\}); \Gamma, x : \alpha \models e \Rightarrow ty_1 \mid Cs_1}{R; \Gamma \models \lambda x. e \Rightarrow \alpha \rightarrow ty_1 \mid Cs_1}$	
I-PROD $\frac{R; \Gamma \models e_1 \Rightarrow ty_1 \mid Cs_1 \quad R; \Gamma \models e_2 \Rightarrow ty_2 \mid Cs_2}{R; \Gamma \models (e_1, e_2) \Rightarrow ty_1 \times ty_2 \mid Cs_1, Cs_2}$	I-ASCRIBE-MONO $\frac{R; \Gamma \models e \Rightarrow ty_1 \mid Cs_1}{R; \Gamma \models (e : ty_2) \Rightarrow ty_2 \mid Cs_1, ty_1 \sim ty_2}$	
I-COND $\frac{R; \Gamma \models e_1 \Rightarrow ty_1 \mid Cs_1 \quad R; \Gamma \models e_2 \Rightarrow ty_2 \mid Cs_2 \quad R; \Gamma \models e_3 \Rightarrow ty_3 \mid Cs_3}{R; \Gamma \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow ty_2 \mid Cs_1, Cs_2, Cs_3, ty_1 \sim \text{Bool}, ty_2 \sim ty_3}$	I-MATCH $\frac{R; \Gamma \models e \Rightarrow ty_1 \mid Cs_1 \quad \Gamma \vdash pat : ty_1 = \Gamma_1 \quad R; \Gamma \cup \Gamma_1 \models e' \Rightarrow ty_2 \mid Cs_2}{R; \Gamma \models \text{match } e \text{ with } \mid pat \rightarrow e' \Rightarrow ty_2 \mid Cs_1, Cs_2}$	
I-FIX $\frac{\alpha \text{ fresh} \quad R; \Gamma, f : \alpha \models e \Rightarrow ty_1 \mid Cs_1}{R; \Gamma \models \text{fix } (\lambda f. e) \Rightarrow \alpha \mid Cs_1, ty_1 \sim \alpha}$		

Function Application. We eagerly consume the arrow when possible. In the case that the function's instantiated type is not (yet) an arrow, possibly a metavariable, we leave an ordinary equation to the pool. A forall that surfaces in it only after solving can only unify with another forall – it is never instantiated (see Instantiation). A rank-n function hiding behind a metavariable thus needs an ascription guiding it, e.g. with $g : \text{Int} \rightarrow (\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \text{Int}$, **apply g 1 id** needs $(\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \text{Int}$ on **apply g 1**. The two cases are given below as

I-APP-ARROW $\frac{R; \Gamma \models e_1 \Rightarrow ty_1 \mid Cs_1 \quad \models^i ty_1 \Rightarrow \alpha \rightarrow \alpha' \mid Cs_2 \quad R; \Gamma \models e_2 \Leftarrow \alpha \mid Cs_3}{R; \Gamma \models e_1 e_2 \Rightarrow \alpha' \mid Cs_1, Cs_2, Cs_3}$	I-APP-MV $\frac{R; \Gamma \models e_1 \Rightarrow ty_1 \mid Cs_1 \quad \models^i ty_1 \Rightarrow \tau_1 \mid Cs_2 \quad \alpha \text{ fresh} \quad R; \Gamma \models e_2 \Rightarrow ty_2 \mid Cs_3}{R; \Gamma \models e_1 e_2 \Rightarrow \alpha \mid Cs_1, Cs_2, Cs_3, \tau_1 \sim (ty_2 \rightarrow \alpha)}$
---	--

Type ascriptions are divided into rank-1 and rank-n. The shallow subsumption $\vdash_{\text{sh}}$ from jones2007 gave rise to two derivation rules SPEC and SKOL which corresponding to the two **solveAlls** in rank-1. Meanwhile, we switch to check mode for rank-n modulo predicates for which the paper's rules GEN2 and ANNOT provide guidances. Monotype ascription is trivial enough that we've given it above already.

Rank-1 Ascription. Rule SPEC solves against the instantiated annotation whereas rule SKOL solves against its skolemization, with escape-check on rigids. It is worth noting the solving here is *lax*, or *best effort*, in the comments, which simply means “solve what we can”: by returning a partial subst, leaving the rest to the outer pool (i.e. the outer call to `solveAll`) to try again instead of signaling an error. Readers may see this as a kind of backtracking – we try again in hopes that the outer pool may reveal more solutions that makes the unsolvables solvable.

$$\begin{array}{c}
\text{I-ASCRIBE-RANK1} \\
\frac{
\begin{array}{c}
R; \Gamma \models e \Rightarrow ty_1 \mid Cs_1 \\
\vdash^i (\forall \overline{tv_i}^i . \bar{\pi} \Rightarrow ty'_1) \Rightarrow ty'_1 \mid Cs_2 \\
ty'_1 \text{ mono} \\
\text{solve}(Cs_1, ty_1 \sim ty'_1) = (\theta_1, \Delta_1, L_1) \\
[\theta_1]\alpha = \alpha \quad \alpha \in R \\
\text{solve}(Cs_1, ty_1 \sim ([\overline{tv_i} \mapsto a_i^i]\tau)) = (\theta_2, \Delta_2, L_2)
\end{array}
}{R; \Gamma \models (e : sch) \Rightarrow ty'_1 \mid Cs_1, Cs_2, ty_1 \sim ty'_1}
\end{array}
\qquad
\begin{array}{c}
\text{I-ASCRIBE-RANKN} \\
\frac{
\begin{array}{c}
\vdash^i sch \Rightarrow \tau_1 \mid Cs_1 \\
R; \Gamma \models e \Leftarrow \tau_1 \mid Cs_2
\end{array}
}{R; \Gamma \models (e : sch) \Rightarrow \tau_1 \mid Cs_1, Cs_2}
\end{array}$$

Rank-N Ascription. An interesting case is that we do not have prenex conversion⁴ that rule GEN2 requires. From what we have gathered, a direct result seems to be, that, e.g. the two supposedly isomorphic types are no longer so,

$$\forall \alpha \beta. \alpha \rightarrow \beta \rightarrow \beta \not\cong \forall \alpha. \alpha \rightarrow \forall \beta. \beta \rightarrow \beta.$$

This used to be papered over by rule U-INST1, which turned out to be unsound and has been removed. Now ascribing the latter to a variable f of the former type is rejected though eta-expansion ($\lambda x. f x$ is accepted) circumvents it – which aligns with GHC’s behavior⁵.

Let-expression. “Let should not be generalized” is a polemic about abolishing local let generalization for type system with complex extensions. While our system isn’t complex enough to justify that, the author’s exposition about generalizing in `OutsideIn(X)` remains relevant to us as well: A to-be-generalized free var shared by both pending predicates and result types would be erroneously monomorphized by solve-time resolution the pool introduces, missing what could be generalized.

$$\begin{array}{c}
\text{I-LET} \\
\frac{
\begin{array}{c}
R; \Gamma \models e \Rightarrow ty_1 \mid Cs_1 \\
\text{solvePure } Cs_1 = (\theta_1, \Delta_1, L_1) \\
\text{rigidize}((\text{fv}([\theta_1]ty_1) \cap \text{fv } \Delta_1) \setminus (\text{fv } \Gamma \cup \text{fv } L_1)) = \theta^{\text{sk}} \\
\text{solve}([\theta^{\text{sk}}]Cs_1) = (\theta_2, \Delta_2, \emptyset) \\
\text{gen } \Gamma (\text{unskolem}([\theta_2 \circ \theta_1]ty_1)) (\text{unskolem}([\theta_2]\Delta_2)) = sch \\
R; \Gamma, x : sch \models e' \Rightarrow ty_2 \mid Cs_2
\end{array}
}{R; \Gamma \models \text{let } x = e \text{ in } e' \Rightarrow ty_2 \mid Cs_2}
\end{array}$$

where L denotes the constraint set that contains *leftover* equations; `rigidize` and `unskolem` kept opaque for brevity.

`OutsideIn(X)` proposes the concept of “untouchables” to bar refinement from touching those variables. While we could do some plumbing in the pool and maintain a so-called *untouchable set*, the

⁴the coercion counterpart of deep skolemization $\vdash_{\text{dsh}}$. The latter is used for *annotated* lambdas rule AABS2 which is absent here at syntax level. Though we need them back if we were to support OCaml’s locally abstract types or GHC’s scoped type variables. Not a real concern at the moment.

⁵The author verified this claim on his machine with GHC 9.14.1.

straightforward, readily available tool in our system is *skolemization*. Thus, typing a Let-expression requires the following steps:

1. Solves the group's equations to exhaustion with refinement disabled. Which always makes sense whether we take predicate constraint into consideration or not.
2. Skolemizes the abstract-able⁶ free vars on top of the substituted result from step 1.
3. Re-solve again with refinement enabled.
4. Manually consume the group's constraint to avoid outer pools re-solving the same, unskolemized constraints, otherwise it defeats the very work we've done. Given the fact that the pool is non-consuming in the sense it does not consumes constraints by itself. Practically, `solveAll` isn't even in the typechecking monad `InferC`, pure-by-construction.
5. Finally, undo step 2 (unskolemize) and generalize by plain `HM(X)`.

Checking. Checking monotypes is trivial. It is more interesting to see what unrestricted binding under checking allows us to achieve.

$R; \Gamma \models e \Leftarrow ty \mid Cs$		<i>(term checking)</i>
C-SKOL $\frac{\begin{array}{c} a_i \text{ fresh} \\ R; \Gamma \models e \Leftarrow [tv_i \mapsto a_i^i] \tau \mid Cs_1 \end{array}}{R; \Gamma \models e \Leftarrow \forall \overline{tv_i}^i. \bar{\pi} \Rightarrow \tau \mid Cs_1}$	C-DEFAULT $\frac{\begin{array}{c} R; \Gamma \models e \Rightarrow ty_1 \mid Cs_1 \\ \vdash^i ty_1 \Rightarrow \tau_1 \mid Cs_2 \end{array}}{R; \Gamma \models e \Leftarrow ty_2 \mid Cs_1, Cs_2, \tau_1 \sim ty_2}$	
C-LAM-MONO $\frac{\begin{array}{c} x \notin \text{dom}(\Gamma) \quad ty_1 \text{ mono} \\ R; \Gamma, x : ty_1 \models e \Leftarrow ty_2 \mid Cs_1 \end{array}}{R; \Gamma \models \lambda x. e \Leftarrow ty_1 \rightarrow ty_2 \mid Cs_1}$	C-LAM-POLY $\frac{\begin{array}{c} x \notin \text{dom}(\Gamma) \\ R; \Gamma, x : \forall \overline{tv_i}^i. \tau_1 \models e \Leftarrow ty_2 \mid Cs_1 \end{array}}{R; \Gamma \models \lambda x. e \Leftarrow \forall \overline{tv_i}^i. \tau_1 \rightarrow ty_2 \mid Cs_1}$	

Rule C-SKOL is also a derivation of the shallow subsumption. This way we can make sure the type of e is as general (if not more) as that of the user.

Rules LAM-MONO and LAM-POLY are the keys to arbitrary-rank types. Concretely in rule LAM-POLY the parameter is bound with the domain as-is, a scheme, so that a polymorphic parameter instantiates (rule I-VAR) at each use in the body.

Impredicativity, on the other hand, comes with the representation. Since we allow a metavariable to bind anything (polytypes, specifically), a type variable may be instantiated with a polytype – the impredicativity, by definition, of System F. For example, consider

$$\begin{aligned} \text{run} &: (\forall s. s \rightarrow s) \rightarrow \text{Int}, \\ \text{id run} &: ((\forall s. s \rightarrow s) \rightarrow \text{Int}) \rightarrow ((\forall s. s \rightarrow s) \rightarrow \text{Int}), \\ \text{Cons} &: (\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \text{List } (\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \text{List } (\forall \alpha. \alpha \rightarrow \alpha), \end{aligned}$$

where we instantiated quantified TVs with polytypes. Note the difference: a polytype at the top is aliased to a type variable only from a type, as in the latter, never from a term, since rules I-VAR and C-DEFAULT instantiate it first – so much for *free*. However, such a solution is reached by

⁶The exact condition is given in the rules; A crucial point is that we must not quantify over free vars in L since they are determined by the pool, not *robbed* by us; Also, “abstract” and “generalize” are used interchangeably.

equations alone (rules U-VAR-BIND and U-TSCH), and a polytype that is only known after solving – hidden behind a metavariable when the term was inferred or checked – stays opaque. It is neither instantiated nor skolemized, and becomes usable only once it is syntactically visible again: through let-generalization, by substituting it in, or through an ascription; Plain program would get rejected⁷. Polytypes in constructor fields, e.g. $\text{PCons} : (\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \text{PList} \rightarrow \text{PList}$, are unaffected, since the field's type is visible both when constructing and when matching.

Notice the claim is quantified with *partial*. The deleted rule U-INST1 instantiated any forall against non-polys in an equation thus a polytype behind a metavariable *was* instantiated at its uses: `(id run) id, apply g 1 id, [id] : List (forall alpha. alpha -> alpha)`, and matching an element out of such a list to apply it all typechecked. That part of impredicativity was obtained erroneously which we shall explain below.

Instantiation. A central part of our typechecker, which, is especially the case after first-order refactoring, unification is no longer higher-order – homogenous unification for polytypes up to renaming by rule U-TSCH. Instantiation thus happens in rules I-VAR, I-APP-ARROW, and C-DEFAULT and rule SPEC of rank-1 ascription, where we know the provided and expected types.

Remark (Why rule U-INST1 is unsound). There used to be one more instantiation, rule U-INST1, inside unification: a scheme $\forall \overline{tv_i}^i. ty$ against a monotype ty' instantiated $\overline{tv_i}^i$ with fresh metavariables and unified the body with ty .

$$\text{U-INST1} \quad \frac{\beta_i \text{ fresh} \quad \text{unify}([\overline{tv_i} \mapsto \beta_i^i] \tau_1) \quad ty_1 = \theta_1}{\text{unify} \forall \overline{tv_i}^i. \pi \Rightarrow \tau_1 \quad ty_1 = \theta_1}$$

It was meant for rank-n functions hiding behind a metavariable e.g. `apply g 1 id`, on the grounds that skolemizing instead would reject $\forall \alpha. \alpha \rightarrow \alpha \sim \beta \rightarrow \text{Bool}$, which solves with $\alpha \mapsto \text{Bool}$. That argument – $\vdash \iota \sigma \Rightarrow \tau$ holds only for

1. a poly on the *provided* side,
2. the check is at the top of that provided/expected comparison.

but binrel ($\sim$) is commutative – equation commutes and rule U-INST1 isn't guaranteed to be at the top – we satisfies neither. Specifically, unification is symmetric (though not shown, the rules are implicitly commutative), and rules U-ARR, U-PROD, and U-APP recurse into arrow domains first (where the direction flips) and type arguments before the rule is reached. To put it simply, rule U-INST1 instantiated polytypes that *should've stayed poly* – the solution (substs) need not equate both sides – the one thing unification promises. If we were to keep it, we would unsoundly accept:

- $h : (\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int}$ aliased to $k : (\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \text{Int}$, rank-2 only: k applies its argument to `true`, and h passes it a `(1 + _)`;
- `apply g 1 (lambda x. x + 1)`, the very case the rule was for, with a g that applies its argument to `true`;
- `x ++ y` where $x : \text{List}(\text{Int} \rightarrow \text{Int})$, $y : \text{List}(\forall \alpha. \alpha \rightarrow \alpha)$: the pool first solves the metavariable of `(++)` to the polytype, then the rule accepts $\text{List}(\text{Int} \rightarrow \text{Int}) \sim \text{List}(\forall \alpha. \alpha \rightarrow \alpha)$ under `List`.

It is now removed at the expense of impredicativity discussed under Checking⁸.

⁷see `examples/impred` for concrete programs.

⁸`error/{rankn-{contra,mv},impred-share}` have more counterexamples.